Meeting 13. Solving Linear Systems

The walk · Lines, planes, and the condition number · Elimination, three ways · Residual and error · Sparse matrices · Fill-in · Solving by iterating

A **linear system** is a set of linear equations that must hold at the same time. We will represent and solve these systems, assess numerical accuracy, and use sparse matrices when most coefficients are zero. A random walk provides an application: from a given node of a graph, what is the probability of reaching a red node before a blue one?

The first notebook covers linear systems, their geometry, and the random walk. The second compares solvers, examines round-off, and introduces sparse storage. The interactive pages below illustrate these topics, including elimination fill-in and iterative solution methods. Timings identified as local measurements come from a machine with 32 logical processors; Colab measurements come from the class's discussion posts.

The notebooks, the pages and the decks of this meeting.

Each notebook opens in Google Colab. Choose *Save a copy in Drive* in the *File* menu before you change anything, to save your work.

- The first notebook: linear systems Parts 1 to 4: two unknowns and two lines, three unknowns and three planes, nearly parallel lines and the condition number, a walk on a graph.
- The second notebook: solvers and sparse matrices Parts 1 to 6: elimination with loops, three ways to solve, round-off and the condition number, the formats COO, CSR and CSC, a grid solved dense and sparse, the product as the matrix fills.

Interactive demonstrations:

- A walk on a graph simulate walks on the notebook's graph and compare their results with probabilities computed from a linear system
- Linear systems, in pictures two lines, nearly parallel lines, three planes that turn, and elimination as planes that pivot
- Solvers, round-off and sparse matrices, in motion compare solvers, examine residuals and errors, explore sparse formats and fill-in, and run Kaczmarz projections

Companion slide decks, with explanatory notes and PDF versions:

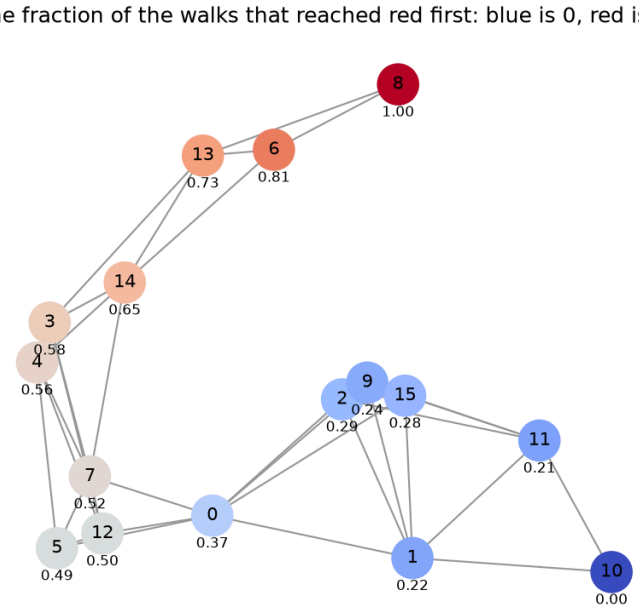
- Lines, Planes, and a Walk Recap 1 (PDF)
- Solvers, Round-off, and Sparse Matrices Recap 2 (PDF)
- Elimination and Fill-in A deeper look 1 (PDF)
- Solving by Iterating A deeper look 2 (PDF)

The main points

The following ten ideas summarize the meeting. Each links to the explanation and the code.

1 Random walks estimate hitting probabilities

A **graph** consists of **nodes** joined by **edges**. It can represent web pages and links, or people and friendships. In the notebook, a particle starts at a node and repeatedly moves to a uniformly chosen neighbour until it reaches a marked red or blue node. The fraction of 2,000 walks that reach red first estimates that starting node's hitting probability. The estimate varies between simulations, and individual walks can take different numbers of steps.



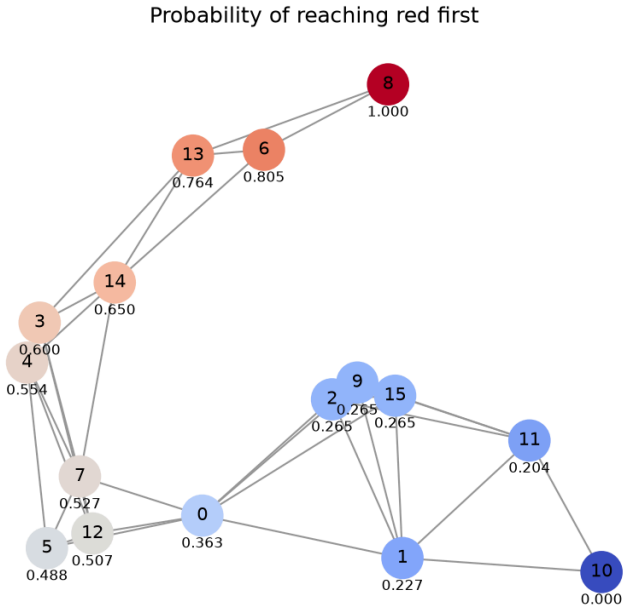
The notebook's graph: red is node 8, blue is node 10. Each node is coloured by the fraction of its 2,000 walks that reached red first.

Section 1.4.

2 Hitting probabilities satisfy a linear system

Let x_v be the probability that a walk from node v reaches red first. The boundary values are 1 at red and 0 at blue. At any other node, conditioning on the first step gives the **average of the neighbours' probabilities**. For example, node 15 has neighbours 0, 1, 2, 9 and 11, so $x_{15} = (x_0 + x_1 + x_2 + x_9 + x_{11}) / 5$. Together with the boundary equations, these form sixteen linear equations in sixteen unknowns.

A direct solve gives approximately 0.363 for node 0 and 0.227 for node 1. The largest difference from the simulation estimates was 0.032. Solving this small system avoids sampling error; the computed probabilities still have floating-point rounding error.



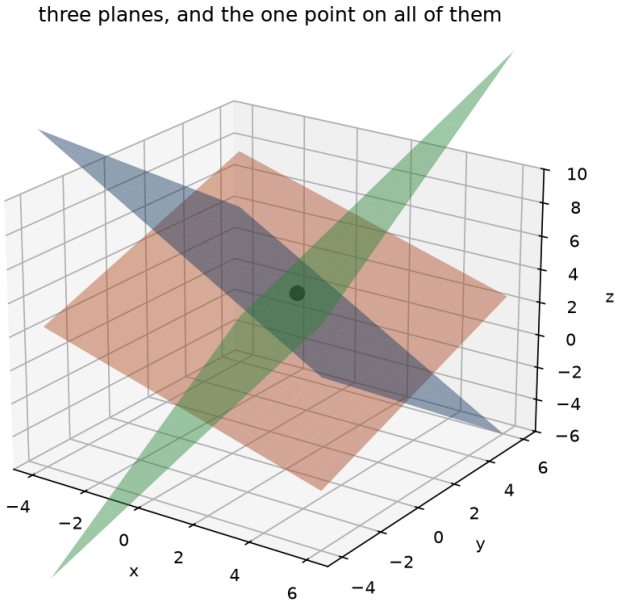
Probabilities computed by solving the linear system.

Section 1.4.

3 Solutions are intersections of lines or planes

A nondegenerate linear equation in two unknowns describes a **line**. A solution of two equations lies on both lines. In three dimensions, the corresponding objects are **planes**; in higher dimensions they are hyperplanes. The rows of `A` contain the equation coefficients, and `A @ x` evaluates all the left-hand sides.

The **rank** of `A` is the number of independent coefficient rows. A square system with full rank has a unique solution. In the rank-2 example, the third plane contains the intersection line of the first two, giving infinitely many solutions. Shifting that third plane parallel to itself makes the system inconsistent. `np.linalg.solve` reports *Singular matrix* for both examples.

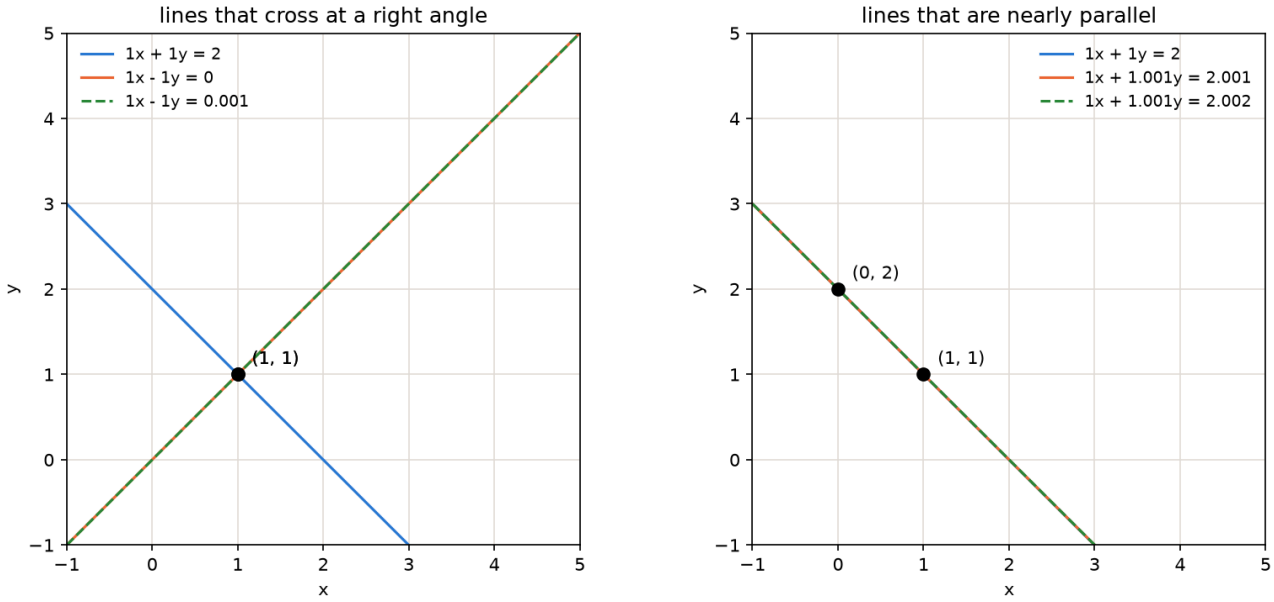


Three planes, and the one point on all of them: $x = 1$, $y = 2$, $z = 3$.

Sections 1.1 and 1.2.

4 Nearly parallel lines: the condition number

The **condition number**, computed by `np.linalg.cond(A)`, bounds the amplification of relative changes in the right-hand side. Nearly parallel lines illustrate high sensitivity: a small shift of one line can move their intersection substantially. In the notebook, changing one right-hand side by 0.001 moves the solution by about 1.4, and the matrix has condition number about 4,000. The comparison system has condition number 1. With float64 arithmetic, a condition number of 10^k can cost roughly k of the available sixteen decimal digits. Actual accuracy depends on the system and its errors.



The same change of a thousandth, at a right angle and at a small angle.

Section 1.3.

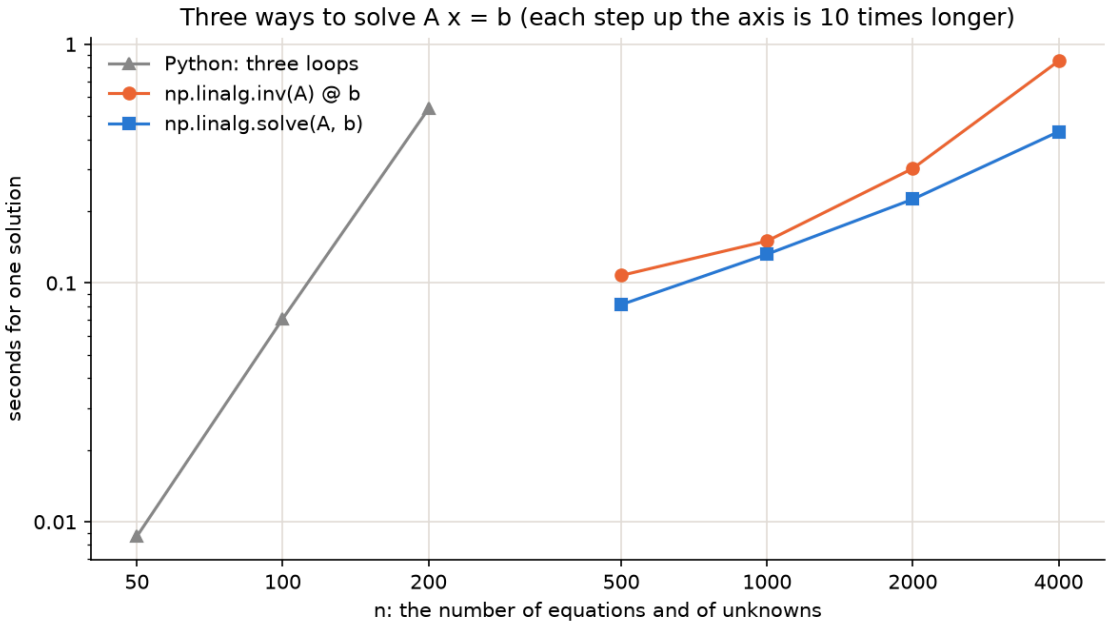
5 Gaussian elimination takes O(n³) arithmetic operations

Gaussian elimination subtracts multiples of equations to remove unknowns while preserving the solution set. It produces an upper-triangular system, which **back substitution** solves from the last unknown to the first. The innermost matrix update runs about $n^3 / 3$ times and uses one multiplication and one subtraction. The leading arithmetic count is therefore about $2n^3 / 3$, or $\mathbf{O(n^3)}$. Doubling n gives approximately eight times the work.

Section 2.1.

6 Solving directly and forming an inverse

In the class's Colab measurements, Python elimination took about 1.4 seconds for 200 unknowns. Cubic extrapolation gives roughly three hours for 4,000 unknowns. At that larger size, `np.linalg.inv(A) @ b` took about 6 seconds and `np.linalg.solve(A, b)` about 1.5 seconds. Forming an entire inverse requires additional work beyond solving for one right-hand side. For $Ax = b$, use `solve`; the precise timing advantage depends on the implementation and hardware.

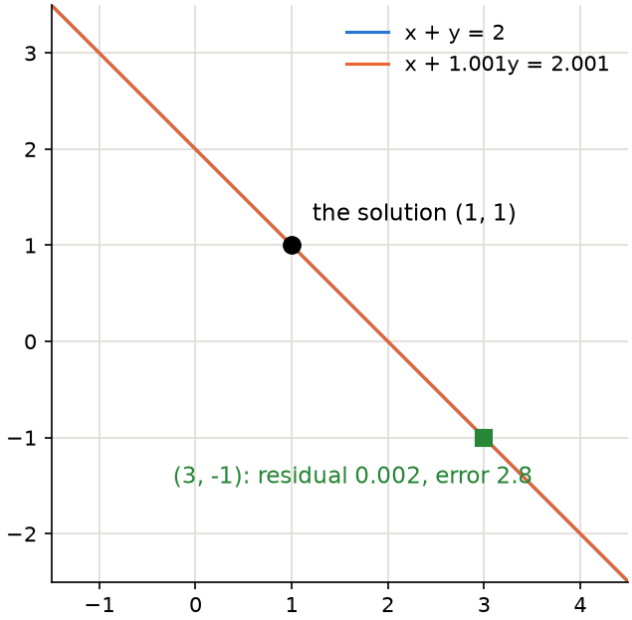


The three ways, timed on the machine that built these notes.

Section 2.2.

7 Residual, solution error, and conditioning

For a computed solution $\hat{x}$, the **residual** is $b - A\hat{x}$. It can be evaluated from the available data. The **solution error**, $\hat{x} - x$, requires the true solution x, which is usually unknown. For a nonsingular system, relative solution error is bounded by the condition number times the relative residual, using the same norm. A small residual can therefore accompany a large error when the matrix is ill-conditioned. The notebook uses a known solution to measure both quantities and compare them.

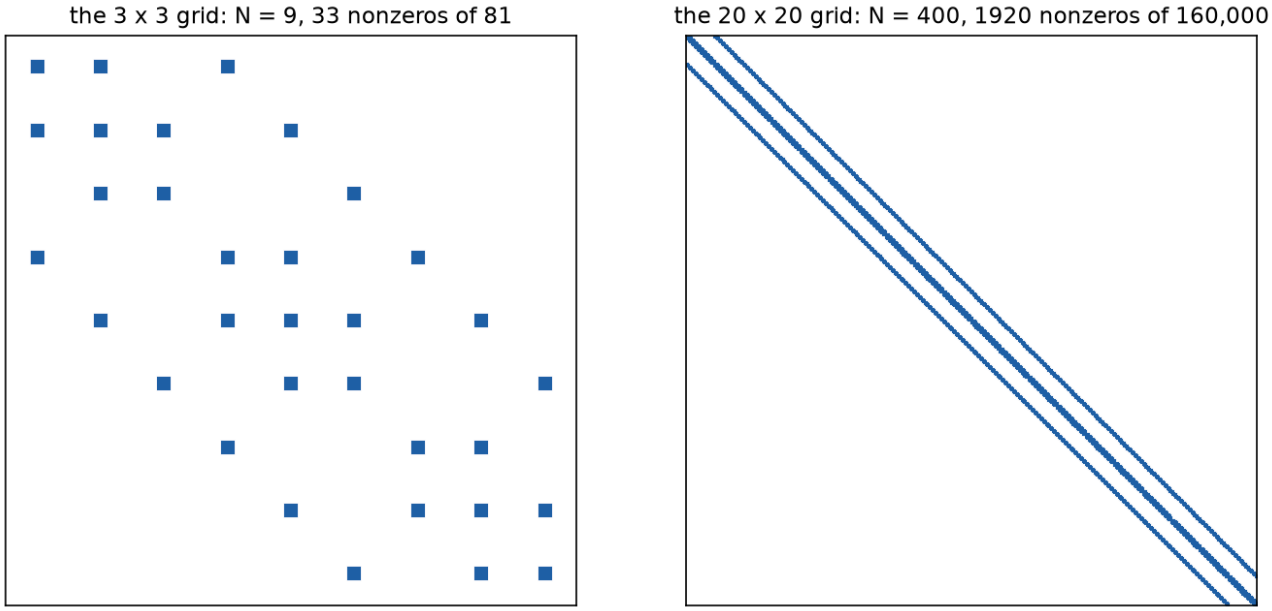


A candidate close to both lines can be far from their intersection. The labels give absolute norms.

Section 2.3.

8 Sparse storage and matrix-vector multiplication

A **sparse matrix** has relatively few nonzero entries. Sparse formats store their values and positions: **COO** lists row, column, and value; **CSR** groups entries by row and records row boundaries; **CSC** groups them by column. In `A @ v`, a zero entry contributes nothing, so sparse multiplication can skip it. For an N-row CSR matrix, the work is $\mathbf{O(nnz + N)}$, including the output vector. For the grid matrices here, every row has nonzeros, so this simplifies to $\mathbf{O(nnz)}$.

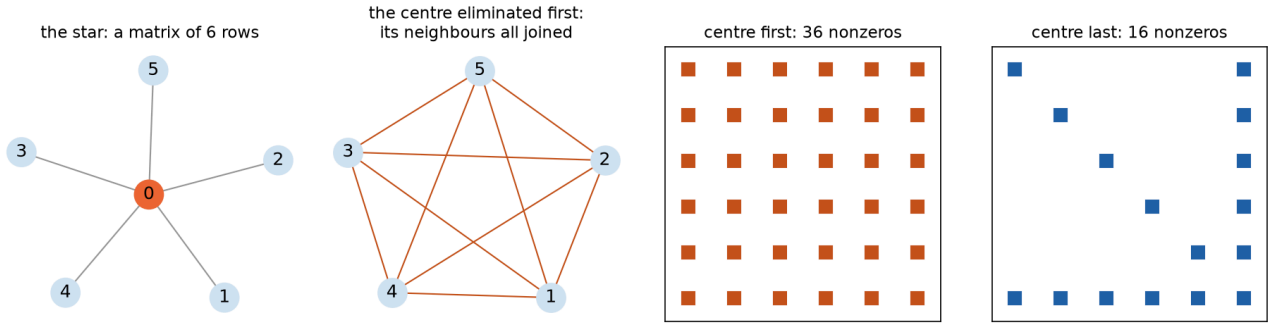


Nonzero entries of the grid matrices. Blank positions represent zeros.

Sections 2.4 to 2.6.

9 Elimination order affects fill-in

Elimination can create new nonzero entries, called **fill-in**. For a symmetric matrix, its graph shows this as new edges among the remaining neighbours of an eliminated node. Eliminating the centre of a star first connects every pair of leaves. Eliminating the leaves first creates no fill. Orderings such as **minimum degree** and **nested dissection** aim to reduce the storage and arithmetic required by the factors. For a two-dimensional grid with N unknowns, nested dissection gives $O(N \log N)$ factor nonzeros and $O(N^{1.5})$ arithmetic work.

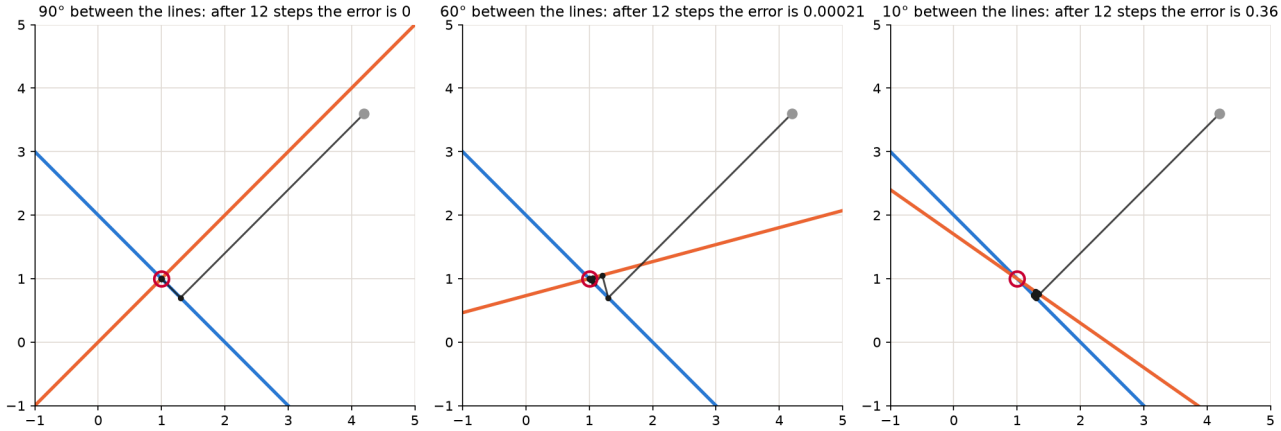


Centre-first elimination connects all five leaves. Eliminating the leaves first adds no fill.

Section 3.

10 Solving by iterating: project onto one equation at a time

An **iterative method** starts from an approximate solution and updates it until a stopping criterion is met. Kaczmarz's method projects the current point onto one equation's line or plane, then processes another equation. For a consistent system, cycling through all rows approaches a solution. With two lines, convergence is fast at a wide angle and slow when the lines are nearly parallel. A sparse step processes only the nonzeros in the selected row. The required number of steps depends on the system, row order, tolerance, and numerical precision.



Twelve projections from the same start, with the lines at 90, 60 and 10 degrees.

Section 4.

Working with the notebooks, and with an AI assistant.

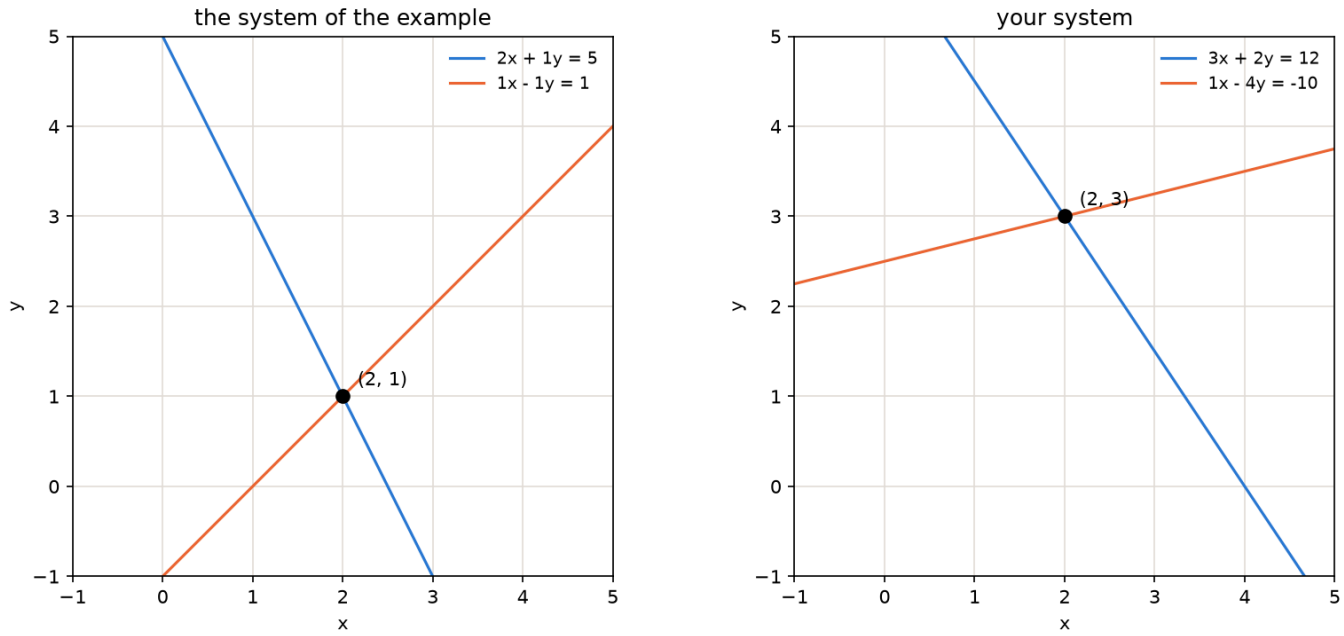
Run each notebook from the top with Shift+Enter. Complete each **Your turn** exercise, then run its check. Part 4 of the second notebook asks you to have an AI assistant write a function. Put your request in **Your prompt**, specifying the function's name, argument, return values, and allowed libraries. Write the **Your thinking** responses in your own words.

The first notebook. Linear systems Open the first notebook

1.1 Two unknowns, two equations Part 1

A **linear equation** is an equality between a sum of unknowns multiplied by fixed coefficients and a right-hand side. A **linear system** requires several such equations to hold together. To solve $2x + y = 5$ and $x - y = 1$ by elimination, add the equations: $3x = 6$, so $x = 2$. Substitution then gives $y = 1$.

In matrix form, `A` contains the coefficients, one row per equation; `x` contains the unknowns; and `b` contains the right-hand sides. The product `A @ x` evaluates the left-hand sides. `np.linalg.solve(A, b)` computes a numerical solution of $Ax = b$ for a square, nonsingular matrix. The exercise asks you to enter $3x + 2y = 12$ and $x - 4y = -10$ as `A2` and `b2`. Its check detects transposed coefficients and missing signs. The next cell plots both systems.



Each equation is a line; each solution is a crossing.

The thinking cell extends the notation to a thousand equations: describe what one row of `A`, one entry of `b`, and the product `A @ x` represent.

Code: Part 1, the system solved, the system typed, its check, and the picture — cells `solve-two`, `paste-system`, `part1-check`, `lines-picture`

```
A = np.array([[2.0, 1.0],
              [1.0, -1.0]])        # one row for each equation: the numbers that multiply x and y
b = np.array([5.0, 1.0])           # the right-hand sides
x = np.linalg.solve(A, b)
print("the solution x:", x)
print("A @ x:", A @ x, "  which is b again")
```

OUTPUT

```
the solution x: [2. 1.]
A @ x: [5. 1.]   which is b again
```

```
A2 = np.array([[3.0, 2.0],
               [1.0, -4.0]])
b2 = np.array([12.0, -10.0])
```

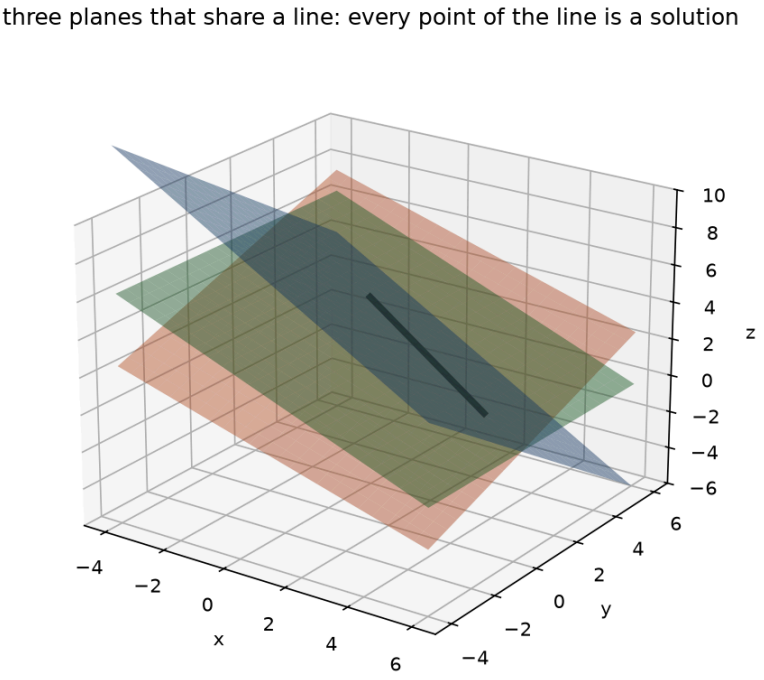
OUTPUT

```
Correct: the solution is [2. 3.] and A2 @ x gives b2 back.
```

1.2 Three unknowns, three planes Part 2

The notebook's interactive plot shows three planes intersecting at $(1, 2, 3)$. The first two intersect along a line, and the third intersects that line at one point. Replacing the third equation by the sum of the first two, $2x + 0y + 3z = 11$, makes its plane contain the entire line. Every point on the line is then a solution.

The matrix has **rank** 2: only two coefficient rows are independent. `np.linalg.solve` reports *Singular matrix*. Changing only the third right-hand side from 11 to 12 shifts that plane away from the intersection line. The system then has no solution, but the same singular coefficient matrix. In two dimensions, coincident lines give infinitely many solutions and distinct parallel lines give none.



The third equation is the sum of the first two: the planes share a line, and the rank is 2.

Code: Part 2, the three planes, the third row, and the check that reads the error cells `planes`, `paste-singular`, `part2-check`

```
A3 = np.array([[1.0, 1.0, 1.0],
               [1.0, -1.0, 2.0],
               [2.0, 1.0, -1.0]])
b3 = np.array([6.0, 5.0, 1.0])
x3 = np.linalg.solve(A3, b3)
print("the solution:", x3, "  A3 @ x3:", A3 @ x3)
draw_planes(A3, b3, point=x3, title="three planes, and the one point on all of them")
```

OUTPUT

```
the solution: [1. 2. 3.]   A3 @ x3: [6. 5. 1.]
```

```
third_row = [2.0, 0.0, 3.0]
third_rhs = 11.0
```

OUTPUT

```
NumPy refuses the system: Singular matrix
Correct: the three equations hold only 2 independent ones, and the planes share a line.
```

Full rank does not imply good conditioning. As the third plane approaches the intersection line of the first two, the system can remain rank 3 while becoming increasingly sensitive. Rank drops only when an exact dependency is reached.

1.3 When the lines are nearly parallel Part 3

The two example systems both have solution $(1, 1)$. In one, the lines meet at a right angle. In the other, $x + y = 2$ and $x + 1.001y = 2.001$, they are nearly parallel. Increasing the second right-hand side by 0.001 moves the solutions by 0.0007 and 1.4142, respectively. Their condition numbers are 1 and 4,002.

The condition number bounds amplification of *relative* changes in b. The displacements above are absolute distances, so their ratio is a different quantity. A condition number of 10^k can cost roughly k decimal digits of accuracy. The exercise asks for a second row that gives a condition number of at least a million while keeping the matrix nonsingular.

Code: Part 3, the two systems nudged, the ill-conditioned row, and its check cells `nearly-parallel`, `paste-cond`, `part3-check`

```
A_good = np.array([[1.0, 1.0], [1.0, -1.0]])
A_bad = np.array([[1.0, 1.0], [1.0, 1.001]])
x_true = np.array([1.0, 1.0])
change = np.array([0.0, 0.001])                    # a change of one thousandth in the second right-hand side

fig, axes = plt.subplots(1, 2, figsize=(11, 5))
for ax, A_, name in ((axes[0], A_good, "lines that cross at a right angle"), (axes[1], A_bad, "lines that are nearly
parallel")):
    b_ = A_ @ x_true
    x_moved = np.linalg.solve(A_, b_ + change)
    print(f"{name}:")
    print(f"   the solution {np.linalg.solve(A_, b_)}, after the change {x_moved}: moved by {np.linalg.norm(x_moved -
x_true):.4f}")
    print(f"   condition number {np.linalg.cond(A_):,.0f}")
    draw_lines(ax, np.vstack([A_, A_[1]]), np.append(b_, (b_ + change)[1]), name, points=[x_true, x_moved], dashed=(2,))
fig.tight_layout()
plt.show()
```

OUTPUT

```
lines that cross at a right angle:
   the solution [1. 1.], after the change [1.0005 0.9995]: moved by 0.0007
   condition number 1
lines that are nearly parallel:
   the solution [1. 1.], after the change [0. 2.]: moved by 1.4142
   condition number 4,002
```

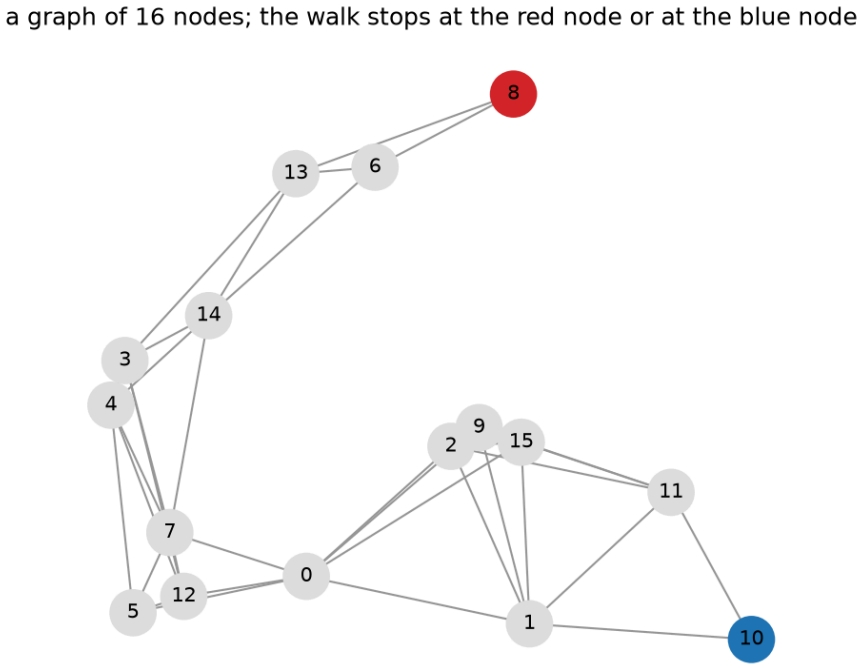
```
second_row = [1.0, 1.000001]
```

OUTPUT

```
Correct: the condition number is 4,000,002. A change of 0.001 in b moves the solution from [1, 1] to [-999.
1001.].
```

1.4 A walk on a graph Part 4

The notebook creates a connected graph of sixteen nodes and marks two nodes at maximum graph distance as red and blue. From each starting node, it simulates 2,000 walks, choosing a neighbour uniformly at each step and stopping at red or blue. The fraction reaching red first estimates that node's probability; sampling differences of a few hundredths are plausible at this sample size. The thinking cell asks which estimates are nearest to one half and to 1, and how a node's probability relates to its neighbours' probabilities.



The graph of the walk: red is node 8, blue is node 10.

Code: the graph, simulated walks, and direct solution cells `graph` , `walk` , `rc-walk-solve`

```
G = nx.random_geometric_graph(16, 0.4, seed=7)       # 16 nodes placed at random; an edge joins two nodes closer than 0.4
positions = nx.get_node_attributes(G, "pos")
lengths = dict(nx.all_pairs_shortest_path_length(G))
RED, BLUE = max(((u, v) for u in G for v in G if u < v), key=lambda pair: lengths[pair[0]][pair[1]])  # the two farthest
apart
neighbours = {v: list(G.neighbors(v)) for v in G}

print(f"{G.number_of_nodes()} nodes, {G.number_of_edges()} edges. Red: node {RED}. Blue: node {BLUE}.")
print("the neighbours of each node:", {v: neighbours[v] for v in sorted(G)})
fig, ax = plt.subplots(figsize=(7, 6))
node_colours = ["#d62728" if v == RED else "#1f77b4" if v == BLUE else "#dddddd" for v in G]
nx.draw(G, positions, ax=ax, node_color=node_colours, with_labels=True, node_size=520, font_size=10,
edge_color="#999999")
```

```
ax.set_title("a graph of 16 nodes; the walk stops at the red node or at the blue node")
plt.show()
```

OUTPUT

```
16 nodes, 37 edges. Red: node 8. Blue: node 10.
the neighbours of each node: {0: [1, 2, 5, 7, 9, 12, 15], 1: [0, 2, 9, 10, 11, 15], 2: [0, 1, 9, 11, 15], 3:
[4, 7, 12, 13, 14], 4: [3, 5, 7, 12, 14], 5: [0, 4, 7, 12], 6: [8, 13, 14], 7: [0, 3, 4, 5, 12, 14], 8: [6,
13], 9: [0, 1, 2, 11, 15], 10: [1, 11], 11: [1, 2, 9, 10, 15], 12: [0, 3, 4, 5, 7], 13: [3, 6, 8, 14], 14: [3,
4, 6, 7, 13], 15: [0, 1, 2, 9, 11]}
```

```
def red_first(start, walks=2000):
    """The fraction of `walks` random walks from `start` that reach the red node before the blue node."""
    reached_red = 0
    for _ in range(walks):
        v = start
        while v != RED and v != BLUE:
            v = neighbours[v][rng.integers(len(neighbours[v]))]      # a neighbour chosen at random
        reached_red += v == RED
    return reached_red / walks


fractions = {v: red_first(v) for v in sorted(G)}
print("the fraction of the walks that reached red first, from each node:")
print(", ".join(f"{v}: {fractions[v]:.2f}" for v in fractions))

fig, ax = plt.subplots(figsize=(7, 6))
nx.draw(G, positions, ax=ax, node_color=[fractions[v] for v in G], cmap="coolwarm", vmin=0, vmax=1,
        with_labels=True, node_size=520, font_size=10, edge_color="#999999")
for v in G:
    ax.annotate(f"{fractions[v]:.2f}", positions[v], xytext=(0, -18), textcoords="offset points", ha="center",
fontsize=8)
ax.set_title("the fraction of the walks that reached red first: blue is 0, red is 1")
plt.show()

print("Line for the discussion board:")
print("Red first, from each node: " + ", ".join(f"{v}: {fractions[v]:.2f}" for v in fractions)
      + f". Red is node {RED}, blue is node {BLUE}.")
```

OUTPUT

```
the fraction of the walks that reached red first, from each node:
0: 0.37, 1: 0.22, 2: 0.29, 3: 0.58, 4: 0.56, 5: 0.49, 6: 0.81, 7: 0.52, 8: 1.00, 9: 0.24, 10: 0.00, 11: 0.21,
12: 0.50, 13: 0.73, 14: 0.65, 15: 0.28
Line for the discussion board:
Red first, from each node: 0: 0.37, 1: 0.22, 2: 0.29, 3: 0.58, 4: 0.56, 5: 0.49, 6: 0.81, 7: 0.52, 8: 1.00, 9:
0.24, 10: 0.00, 11: 0.21, 12: 0.50, 13: 0.73, 14: 0.65, 15: 0.28. Red is node 8, blue is node 10.
```

```
# The answer to the question of Part 4: one unknown for each node, and one equation for each.
nodes = sorted(G)
index = {v: i for i, v in enumerate(nodes)}
M = np.zeros((len(nodes), len(nodes)))
rhs = np.zeros(len(nodes))
for v in nodes:
    i = index[v]
    M[i, i] = 1.0
    if v == RED:
        rhs[i] = 1.0                                    # the walk is at red: red first, always
```

```
    elif v == BLUE:
        rhs[i] = 0.0                                # at blue: never
    else:
        for u in neighbours[v]:                     # elsewhere: the average of the neighbours
            M[i, index[u]] -= 1.0 / len(neighbours[v])
solution = np.linalg.solve(M, rhs)

example = next(v for v in nodes if v not in (RED, BLUE) and len(neighbours[v]) == 3)
print(f"node {example} has the neighbours {neighbours[example]}: its equation is "
      f"x{example} = (x{neighbours[example][0]} + x{neighbours[example][1]} + x{neighbours[example][2]}) / 3")
print(f"the row of node {example} in the matrix: 1 at column {example}, " + ", ".join(f"{M[index[example], index[u]]:.3f}
at column {u}" for u in neighbours[example]) + ", 0 elsewhere")
print(f"largest difference between the 2,000 walks and the direct solution: {max(abs(fractions[v] - solution[index[v]])
for v in nodes):.3f}")
print()
print(f"{'node':>5s}{'neighbours':>12s}{'simulated':>11s}{'direct':>8s}")
for v in nodes:
    print(f"{v:5d}{len(neighbours[v]):12d}{fractions[v]:11.2f}{solution[index[v]]:8.3f}")

fig, ax = plt.subplots(figsize=(7, 6))
nx.draw(G, positions, ax=ax, node_color=[solution[index[v]] for v in G], cmap="coolwarm", vmin=0, vmax=1,
        with_labels=True, node_size=520, font_size=10, edge_color="#999999")
for v in G:
    ax.annotate(f"{solution[index[v]]:.3f}", positions[v], xytext=(0, -18), textcoords="offset points", ha="center",
fontsize=8)
ax.set_title("Probability of reaching red first")
plt.show()
```

OUTPUT

```
node 6 has the neighbours [8, 13, 14]: its equation is x6 = (x8 + x13 + x14) / 3
the row of node 6 in the matrix: 1 at column 6, -0.333 at column 8, -0.333 at column 13, -0.333 at column 14, 0
elsewhere
largest difference between the 2,000 walks and the direct solution: 0.032

 node  neighbours  simulated  direct
    0           7       0.37   0.363
    1           6       0.22   0.227
    2           5       0.29   0.265
    3           5       0.58   0.600
    4           5       0.56   0.554
    5           4       0.49   0.488
    6           3       0.81   0.805
    7           6       0.52   0.527
    8           2       1.00   1.000
    9           5       0.24   0.265
   10           2       0.00   0.000
   11           5       0.21   0.204
   12           5       0.50   0.507
   13           4       0.73   0.764
   14           5       0.65   0.650
   15           5       0.28   0.265
```

Node 15 has five neighbours. Its first step selects each with probability $1 / 5$, so its red-first probability is the average of theirs. Applying this rule to each non-boundary node, with red fixed at 1 and blue at 0, gives a linear system. Solving it computes probabilities without sampling walks. Uniform link-following is a modelling assumption; whether it describes actual readers is a separate question.

The second notebook. Solvers and sparse matrices

Open the second notebook

2.1 Elimination, with loops Part 1

Forward elimination processes the unknowns in order. At step k, it subtracts multiples of the **pivot row** k from the rows below, making their column-k entries zero. The divisor `A[k, k]` is the **pivot**. The resulting matrix is **upper triangular**. **Back substitution** then solves for the unknowns from last to first.

The notebook's three loops execute about $n^3 / 3$ inner matrix updates, each containing two arithmetic operations. This simplified implementation does not swap rows; its timing examples are chosen so it works without pivoting. The exercise completes `x[i] = (c[i] - U[i, i + 1:] @ x[i + 1:]) / U[i, i]`. The check detects a missing division, the wrong slice, or a plus sign replacing the minus. At $n = 200$, the local run took 0.537 seconds and a class Colab run took about 1.4 seconds. Cubic extrapolation to 4,000 unknowns gives hours for these Python loops.

Code: Part 1, elimination with three loops, the line of back substitution, its check, and the loops timed cells `elimination`, `paste-back`, `part1-check`, `time-loops`

```
def eliminate(A, b):
    """Forward elimination: a copy of A made upper triangular, and b changed with it, so that the solutions stay the
same."""
    A, b = A.astype(float).copy(), b.astype(float).copy()
    n = len(b)
    for k in range(n):                      # the unknown to remove from the rows below row k
        for i in range(k + 1, n):           # every row below row k
            factor = A[i, k] / A[k, k]      # the multiple of row k that removes the unknown from row i
            for j in range(k, n):           # every column from k on; the columns before k are already 0
                A[i, j] -= factor * A[k, j]
            b[i] -= factor * b[k]
    return A, b


A, b = random_system(4)
U, c = eliminate(A, b)
np.set_printoptions(precision=3, suppress=True)
print("the matrix after elimination, upper triangular:")
print(U)
print("the right-hand side, changed with it:", c)
```

OUTPUT

```
the matrix after elimination, upper triangular:
[[ 4.637  0.27   0.041  0.017]
 [ 0.     4.865  0.599  0.727]
 [ 0.     0.     4.7   -0.134]
 [ 0.     0.     0.     4.196]]
the right-hand side, changed with it: [0.863 0.39  0.126 0.245]
```

```
def back_substitute(U, c):
    """The solution of U x = c for an upper triangular U, from the last unknown to the first."""
```

```
    n = len(c)
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (c[i] - U[i, i + 1:] @ x[i + 1:]) / U[i, i]
    return x
```

OUTPUT

```
Correct: back_substitute finds the solution; eliminate and back_substitute together solve the system.
```

OUTPUT

n	passes, about n^3 / 3	seconds	ratio to the first	nanoseconds per pass
50	41,666	0.009	1.0	209
100	333,333	0.071	8.1	213
200	2,666,666	0.537	61.8	201

Geometrically, subtracting a multiple of one equation from another rotates the changed plane about their shared line and preserves the solution. The approximately $n^3 / 3$ matrix updates each multiply and subtract, giving approximately $2n^3 / 3$ arithmetic operations.

2.2 Three ways to solve Part 2

The experiment compares Python elimination, `np.linalg.inv(A) @ b`, and `np.linalg.solve(A, b)`. Both library approaches have cubic arithmetic cost for general dense matrices, but forming the inverse does additional work beyond solving for one right-hand side. The constant factor depends on the algorithm. `solve` uses compiled elimination with partial pivoting.

At 4,000 unknowns, the local run took 0.850 seconds using the inverse and 0.430 using `solve`. The class's Colab posts reported about 6 and 1.5 seconds. The exercise asks for the inverse-to-solve ratio and the growth in solve time from 2,000 to 4,000 unknowns. Doubling the dimension multiplies the leading operation count by eight; the timing ratio also depends on achieved throughput.

For $Ax = b$, prefer `solve`. It avoids constructing the full inverse, and the next experiment also shows a numerical-accuracy advantage. Sparse inverses can be dense even when the original matrix is sparse. The thinking cell asks which constant factors Big O notation omits.

Code: Part 2, the timing, the two numbers, and their check cells `time-three`, `answers-three`, `part2-check`

```
sizes = [500, 1000, 2000, 4000]
seconds = {"inverse": [], "solve": []}
for n in sizes:
    A, b = random_system(n)
    seconds["inverse"].append(best_time(lambda: np.linalg.inv(A) @ b, repeats=2))
    seconds["solve"].append(best_time(lambda: np.linalg.solve(A, b), repeats=2))

print(f"{'n':>6s}{'np.linalg.inv(A) @ b':>22s}{'np.linalg.solve(A, b)':>24s}{'inverse / solve':>18s}")
for i, n in enumerate(sizes):
    print(f"{n:6d}{seconds['inverse'][i]:22.4f}{seconds['solve'][i]:24.4f}{seconds['inverse'][i] / seconds['solve']
[i]:18.1f}")

fig, ax = plt.subplots(figsize=(8, 4.6))
ax.plot(loop_sizes, loop_times, marker="^", color="#888888", label="Python: three loops")
```

```
ax.plot(sizes, seconds["inverse"], marker="o", color="#eb6834", label="np.linalg.inv(A) @ b")
ax.plot(sizes, seconds["solve"], marker="s", color="#2a78d6", label="np.linalg.solve(A, b)")
ax.set_xscale("log", base=2)
ax.set_yscale("log")
ax.set_xticks(loop_sizes + sizes)
ax.set_xticklabels([str(n) for n in loop_sizes + sizes])
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda value, position: f"{value:g}"))
ax.set_xlabel("n: the number of equations and of unknowns")
ax.set_ylabel("seconds for one solution")
ax.set_title("Three ways to solve A x = b (each step up the axis is 10 times longer)")
ax.grid(True, which="major", color="#e1e0d9", linewidth=0.8)
ax.spines[["top", "right"]].set_visible(False)
ax.legend(frameon=False)
fig.tight_layout()
plt.show()

print("Line for the discussion board:")
print(f"Solving n = 4,000 equations: the inverse {seconds['inverse'][-1]:.3f} s, solve {seconds['solve'][-1]:.3f} s; "
      f"the loops of Python at n = 200: {loop_times[-1]:.2f} s, so about {duration(loop_times[-1] * 8000)} at n =
4,000.")
```

OUTPUT

```
     n  np.linalg.inv(A) @ b   np.linalg.solve(A, b)   inverse / solve
   500                0.1076                  0.0812               1.3
  1000                0.1500                  0.1320               1.1
  2000                0.3019                  0.2244               1.3
  4000                0.8505                  0.4305               2.0
Line for the discussion board:
Solving n = 4,000 equations: the inverse 0.850 s, solve 0.430 s; the loops of Python at n = 200: 0.54 s, so
about 71.6 minutes at n = 4,000.
```

```
INV_OVER_SOLVE = seconds["inverse"][-1] / seconds["solve"][-1]
SOLVE_GROWTH = seconds["solve"][-1] / seconds["solve"][-2]
```

OUTPUT

```
Correct: the inverse takes 2.0 times as long as solve at n = 4,000, and from 2,000 to 4,000 the time of solve
grows by a factor of 1.9.
```

Partial pivoting chooses the largest-magnitude available entry in the current column and swaps its row into position. This numerical choice differs from the ordering used to reduce sparse fill-in.

2.3 Round-off, and the condition number Part 3

Float64 arithmetic provides about sixteen decimal digits of precision. The experiment chooses a known vector `x_true` and constructs `b = A @ x_true` for `A = np.ones((n, n)) + eps * np.eye(n)`, with $n = 100$. For small positive ε, the condition number is about $100 / \varepsilon$. Forming `b` also introduces round-off.

For each computed solution, the table reports **relative solution error**, $\|x - x_{\text{true}}\| / \|x_{\text{true}}\|$, and **relative residual**, $\|Ax - b\| / \|b\|$. The rule of thumb $\max(0, 16 - \log_{10}(\text{cond}(A)))$ estimates the digits that may remain when rounding errors are amplified. It gives about 12, 8, and 4 digits for condition numbers 10^4, 10^8, and 10^{12}; at 10^{16}, it guarantees none. These are estimates, not exact predictions of the observed error.

In this run, the inverse-based result is less accurate, while `solve` retains a relative residual near 10^{-16} even when its solution error is large. The exercise asks for the digit estimate at condition number 10^{10}: about 6. The thinking question asks why points close to two nearly parallel lines can still be far from their intersection.

Code: Part 3, the round-off table, the digits left, and the check cells `roundoff`, `answer-digits`, `part3-check`

```
n = 100
x_true = rng.random(n)
error = lambda x: np.linalg.norm(x - x_true) / np.linalg.norm(x_true)
print(f"{'eps':>8s}{'condition number':>18s}{'error of solve':>16s}{'error of inverse':>18s}{'residual of solve':>19s}
{'residual of inverse':>21s}")
for eps in [1e-2, 1e-6, 1e-10, 1e-14]:
    A = np.ones((n, n)) + eps * np.eye(n)
    b = A @ x_true
    x_solve = np.linalg.solve(A, b)
    x_inverse = np.linalg.inv(A) @ b
    residual = lambda x: np.linalg.norm(A @ x - b) / np.linalg.norm(b)
    print(f"{eps:8.0e}{np.linalg.cond(A):18.1e}{error(x_solve):16.1e}{error(x_inverse):18.1e}"
          f"{residual(x_solve):19.1e}{residual(x_inverse):21.1e}")
```

OUTPUT

eps	condition number	error of solve	error of inverse	residual of solve	residual of inverse
1e-02	1.0e+04	4.7e-13	7.1e-12	1.1e-16	3.2e-13
1e-06	1.0e+08	6.7e-09	9.9e-08	2.1e-16	1.0e-08
1e-10	1.0e+12	5.3e-05	4.1e-03	2.0e-16	1.6e-04
1e-14	1.0e+16	6.3e-01	1.9e+01	3.9e-16	1.0e+00

```
DIGITS_LEFT = 6
```

OUTPUT

```
Correct: about 6 digits, 16 minus 10.
```

Usually the true solution is unavailable, so its error cannot be measured directly. A constructed example can supply a known solution, but in general conditioning is needed to interpret what a small residual says about accuracy.

2.4 Matrices that are mostly zeros Part 4

A **sparse** matrix has relatively few nonzero entries. The example is an $n \times n$ grid with $N = n^2$ unknowns. Its matrix has 4 on the diagonal and -1 for each grid neighbour, giving at most five nonzeros per row. Similar local connections occur in heat-flow and pressure models. A 100 × 100 grid gives a 10,000 × 10,000 matrix: 100 million entries but only 49,600 nonzeros.

COO stores arrays of row indices, column indices, and values. **CSR** stores values and column indices grouped by row, with `indptr` marking each row's boundaries. **CSC** groups entries by column. CSR is convenient for matrix-vector products; CSC is commonly used by sparse direct solvers. The notebook prints all three representations of the 3 × 3 grid matrix.

With float64 values and 32-bit indices, dense storage needs $8N^2$ bytes and CSR needs $12\,\mathrm{nnz} + 4(N+1)$ bytes, including row pointers. For the million-unknown grid, the table gives 8 TB dense and 64 MB in CSR. The exercise asks an AI assistant to write `dense_to_csr` using NumPy without SciPy, then compares its arrays with SciPy's.

Code: Part 4, the grid's matrix in three formats, the function written to the requirements, its check, and the memory table cells `formats`, `paste-csr`, `part4-check`, `memory`

```
def grid_matrix(n):
    """The matrix of an n x n grid in COO format: 4 on the diagonal, -1 for each of the four neighbours of a point."""
    rows, cols, values = [], [], []
    for i in range(n):
        for j in range(n):
            k = i * n + j                           # the point in row i and column j of the grid is unknown k
            rows.append(k)
            cols.append(k)
            values.append(4.0)
            for di, dj in ((-1, 0), (1, 0), (0, -1), (0, 1)):
                if 0 <= i + di < n and 0 <= j + dj < n:
                    rows.append(k)
                    cols.append((i + di) * n + j + dj)
                    values.append(-1.0)
    return sparse.coo_matrix((values, (rows, cols)), shape=(n * n, n * n))


A_coo = grid_matrix(3)
A_csr = A_coo.tocsr()
A_csc = A_coo.tocsc()
np.set_printoptions(linewidth=140)
print("the dense matrix of the 3 x 3 grid, 9 rows and 9 columns:")
print(A_coo.toarray().astype(int))
print(f"nonzeros: {A_coo.nnz} of {A_coo.shape[0] * A_coo.shape[1]} numbers")
print("COO  row    ", A_coo.row)
print("     col    ", A_coo.col)
print("     data   ", A_coo.data.astype(int))
print("CSR  indptr ", A_csr.indptr, "  where each row begins in the two arrays below; the last entry is the number of
nonzeros")
print("     indices", A_csr.indices, "  the column of each nonzero, row by row")
print("     data   ", A_csr.data.astype(int))
print("CSC  indptr ", A_csc.indptr, "  where each column begins")
print("     indices", A_csc.indices, "  the row of each nonzero, column by column")
print("     data   ", A_csc.data.astype(int))
```

OUTPUT

```
the dense matrix of the 3 x 3 grid, 9 rows and 9 columns:
[[ 4 -1  0 -1  0  0  0  0  0]
 [-1  4 -1  0 -1  0  0  0  0]
 [ 0 -1  4  0  0 -1  0  0  0]
 [-1  0  0  4 -1  0 -1  0  0]
 [ 0 -1  0 -1  4 -1  0 -1  0]
 [ 0  0 -1  0 -1  4  0  0 -1]
 [ 0  0  0 -1  0  0  4 -1  0]
 [ 0  0  0  0 -1  0 -1  4 -1]
 [ 0  0  0  0  0 -1  0 -1  4]]
nonzeros: 33 of 81 numbers
COO  row     [0 0 0 1 1 1 1 2 2 2 3 3 3 3 4 4 4 4 4 5 5 5 5 6 6 6 7 7 7 7 8 8 8]
     col     [0 3 1 1 4 0 2 2 5 1 3 0 6 4 4 1 7 3 5 5 2 8 4 6 3 7 7 4 6 8 8 5 7]
     data    [ 4 -1 -1  4 -1 -1 -1  4 -1 -1  4 -1 -1 -1  4 -1 -1 -1 -1  4 -1 -1 -1  4 -1 -1  4 -1 -1 -1  4 -1
-1]
CSR  indptr  [ 0  3  7 10 14 19 23 26 30 33]   where each row begins in the two arrays below; the last entry is
the number of nonzeros
     indices [0 1 3 0 1 2 4 1 2 5 0 3 4 6 1 3 4 5 7 2 4 5 8 3 6 7 4 6 7 8 5 7 8]   the column of each nonzero,
row by row
     data    [ 4 -1 -1 -1  4 -1 -1 -1  4 -1 -1  4 -1 -1 -1 -1  4 -1 -1 -1 -1  4 -1 -1  4 -1 -1 -1  4 -1 -1 -1
4]
CSC  indptr  [ 0  3  7 10 14 19 23 26 30 33]   where each column begins
     indices [0 1 3 0 1 2 4 1 2 5 0 3 4 6 1 3 4 5 7 2 4 5 8 3 6 7 4 6 7 8 5 7 8]   the row of each nonzero,
column by column
     data    [ 4 -1 -1 -1  4 -1 -1 -1  4 -1 -1  4 -1 -1 -1 -1  4 -1 -1 -1 -1  4 -1 -1  4 -1 -1 -1  4 -1 -1 -1
4]
```

```python
def dense_to_csr(A):
    """The three arrays of the CSR format of a dense array: data, indices, indptr."""
    data, indices, indptr = [], [], [0]
    for row in A:
        nonzero = np.nonzero(row)[0]
        data.extend(row[nonzero])
        indices.extend(nonzero)
        indptr.append(len(data))
    return np.array(data, dtype=float), np.array(indices, dtype=int), np.array(indptr, dtype=int)
```

OUTPUT

```
Correct: dense_to_csr gives the same three arrays as scipy.sparse.csr_matrix.
```

OUTPUT

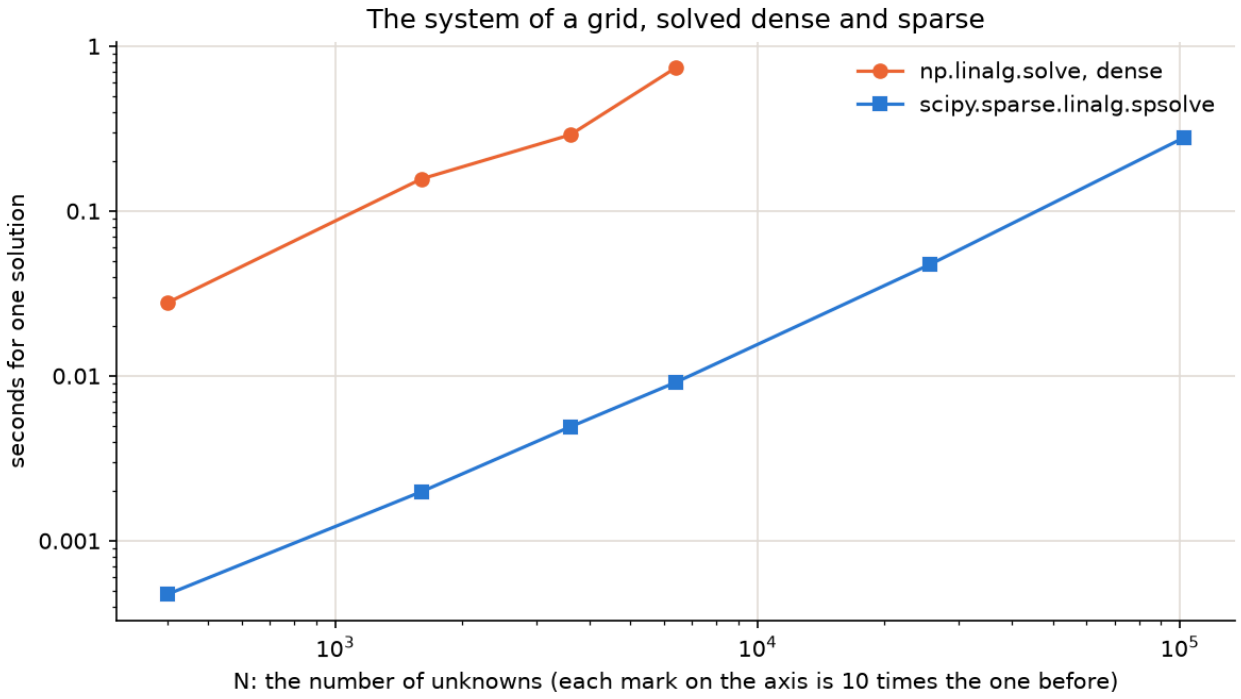
```
    grid  N = unknowns    nonzeros   dense, 8 bytes a number   CSR, 12 bytes a nonzero
   10 x 10            100         460                    80 KB                  5.92 KB
   30 x 30            900       4,380                  6.48 MB                  56.2 KB
  100 x 100        10,000      49,600                   800 MB                   635 KB
  300 x 300        90,000     448,800                  64.8 GB                  5.75 MB
 1000 x 1000    1,000,000   4,996,000                     8 TB                    64 MB
 3000 x 3000    9,000,000  44,988,000                   648 TB                   576 MB
check on the 100 x 100 grid, built: 49600 nonzeros
```

Dense matrix-vector multiplication reads every entry, including zeros. CSR visits stored entries and uses their column indices to select vector values. Irregular indices can make these accesses less predictable than in a grid, so sparse storage does not remove the need to consider memory access patterns.

2.5 Solving, dense and sparse Part 5

`scipy.sparse.linalg.spsolve` works with sparse storage, while `np.linalg.solve` uses a dense array. The experiment compares them on grids with 400 to 6,400 unknowns, then continues with the sparse solver alone to 102,400 unknowns. A dense matrix at that largest size would require about 84 GB.

For the 80 × 80 grid, the local times were 0.739 seconds dense and 0.0091 seconds sparse. Class posts reported about 6 seconds dense and a few hundredths sparse. Dense elimination takes $O(N^3)$ work, so quadrupling N multiplies its leading count by 64. Sparse elimination grows more slowly on these grids, but its cost also depends on the fill-in described in section 3.



The grid's system solved dense and sparse, on the machine that built these notes.

Code: Part 5, the grids solved both ways

cell `time-solvers`

```
grid_sizes = [20, 40, 60, 80, 160, 320]
dense_sizes = [20, 40, 60, 80]                  # the dense matrix of the 80 x 80 grid is 328 MB; the next would be 5 GB
unknowns, dense_seconds_solve, sparse_seconds_solve = [], [], []
for n in grid_sizes:
    A_sparse = grid_matrix(n).tocsc()
    N = n * n
    b = rng.random(N)
    unknowns.append(N)
    sparse_seconds_solve.append(best_time(lambda: sparse_linalg.spsolve(A_sparse, b), repeats=2))
    if n in dense_sizes:
        A_dense = A_sparse.toarray()
        dense_seconds_solve.append(best_time(lambda: np.linalg.solve(A_dense, b), repeats=2))
        del A_dense

print(f"{'grid':>10s}{'N':>9s}{'nonzeros':>10s}{'np.linalg.solve, dense':>25s}{'spsolve, sparse':>18s}")
for i, n in enumerate(grid_sizes):
    dense_text = f"{dense_seconds_solve[i]:.4f} s" if i < len(dense_seconds_solve) else "too large to store"
```

```
    print(f"{n:5d} x {n:<3d}{unknowns[i]:9,d}{grid_matrix(n).nnz:10,d}{dense_text:>25s}{sparse_seconds_solve[i]:16.4f}
s")

fig, ax = plt.subplots(figsize=(8, 4.6))
ax.plot(unknowns[:len(dense_seconds_solve)], dense_seconds_solve, marker="o", color="#eb6834", label="np.linalg.solve,
dense")
ax.plot(unknowns, sparse_seconds_solve, marker="s", color="#2a78d6", label="scipy.sparse.linalg.spsolve")
ax.set_xscale("log")
ax.set_yscale("log")
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda value, position: f"{value:g}"))
ax.set_xlabel("N: the number of unknowns (each mark on the axis is 10 times the one before)")
ax.set_ylabel("seconds for one solution")
ax.set_title("The system of a grid, solved dense and sparse")
ax.grid(True, which="major", color="#e1e0d9", linewidth=0.8)
ax.spines[["top", "right"]].set_visible(False)
ax.legend(frameon=False)
fig.tight_layout()
plt.show()

print("Line for the discussion board:")
print(f"The grid of 80 x 80, N = 6,400: dense {dense_seconds_solve[-1]:.3f} s, sparse {sparse_seconds_solve[3]:.4f} s; "
      f"the grid of 320 x 320, N = 102,400: sparse {sparse_seconds_solve[-1]:.3f} s, dense not possible.")
```

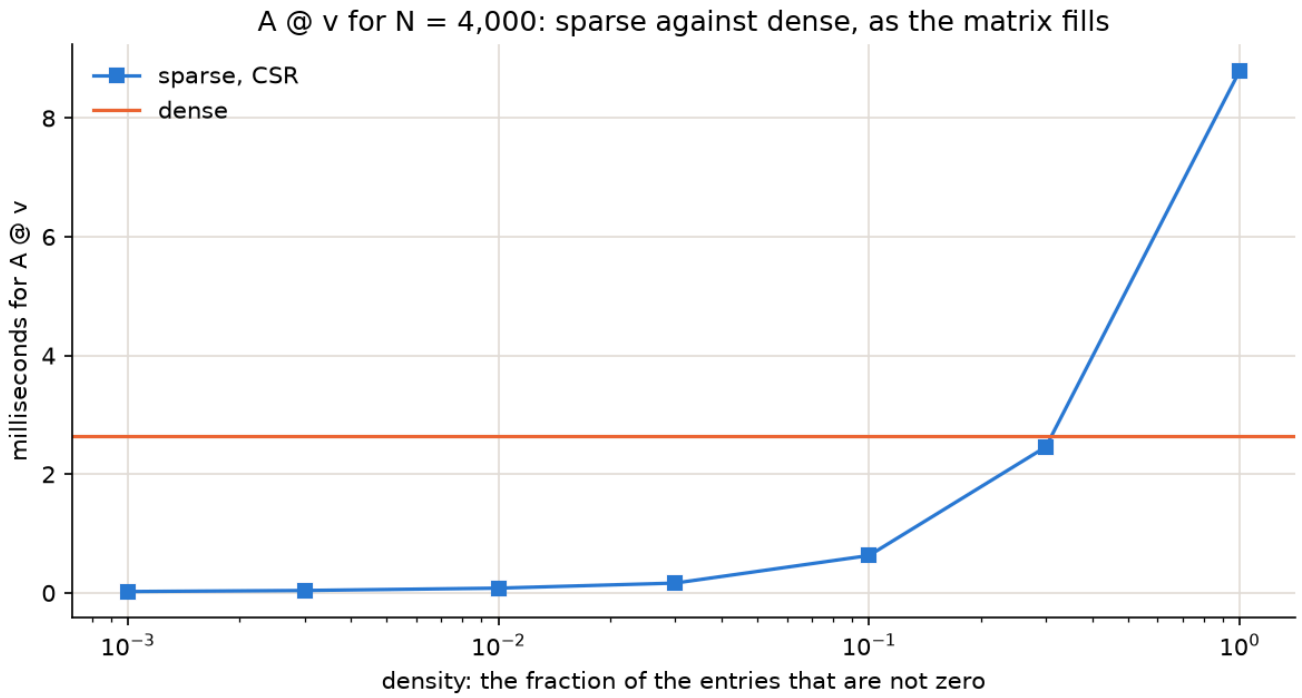
OUTPUT

```
      grid        N  nonzeros   np.linalg.solve, dense   spsolve, sparse
   20 x 20       400     1,920                   0.0277 s          0.0005 s
   40 x 40     1,600     7,840                   0.1565 s          0.0020 s
   60 x 60     3,600    17,760                   0.2897 s          0.0049 s
   80 x 80     6,400    31,680                   0.7390 s          0.0091 s
  160 x 160   25,600   127,360       too large to store          0.0474 s
  320 x 320  102,400   510,720       too large to store          0.2791 s
Line for the discussion board:
The grid of 80 x 80, N = 6,400: dense 0.739 s, sparse 0.0091 s; the grid of 320 x 320, N = 102,400: sparse
0.279 s, dense not possible.
```

2.6 Matrix-vector multiplication at different densities Part 6

A dense $N \times N$ matrix-vector product performs N^2 multiply-add updates. CSR performs one per stored nonzero, plus work to initialize the output. The experiment times random matrices of size 4,000 at densities from 0.001 to 1. It asks for the first tested density at which the dense product is faster, `CROSSOVER_DENSITY`, or `None` if no such row appears.

Sparse access also reads column indices and selects entries from v. These extra accesses can increase the cost per stored entry, so fewer arithmetic operations do not always mean less time. The crossover depends on the machine, libraries, and access pattern. Several Colab groups found CSR faster at every tested density. For the grid, the density is approximately $5 / N$. The final thinking cell asks how the number and arrangement of nonzeros affect the choice of storage.



Dense and CSR matrix-vector timings at different densities, measured locally.

Code: Part 6, the products timed, the crossover, and its check cells `time-products`, `answer-crossover`, `part6-check`

```
N = 4000
v = rng.random(N)
A_dense = rng.random((N, N))
dense_seconds = best_time(lambda: A_dense @ v, repeats=5)
densities = [0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1.0]
sparse_seconds, nonzeros = [], []
for density in densities:
    A_sparse = sparse.random(N, N, density=density, format="csr", random_state=1)
    nonzeros.append(A_sparse.nnz)
    sparse_seconds.append(best_time(lambda: A_sparse @ v, repeats=5))
del A_sparse

print(f"dense product, {N:,} x {N:,} numbers: {dense_seconds * 1e3:.2f} milliseconds, {dense_seconds / N ** 2 * 1e9:.2f}
nanoseconds a number")
print(f"{'density':>9s}{'nonzeros':>12s}{'sparse product, ms':>21s}{'ns a nonzero':>14s}{'faster':>9s}")
for density, nnz, s in zip(densities, nonzeros, sparse_seconds):
    print(f"{density:9g}{nnz:12,d}{s * 1e3:21.2f}{s / nnz * 1e9:14.2f}{'sparse' if s < dense_seconds else 'dense':>9s}")

fig, ax = plt.subplots(figsize=(8, 4.4))
ax.plot(densities, [s * 1e3 for s in sparse_seconds], marker="s", color="#2a78d6", label="sparse, CSR")
ax.axhline(dense_seconds * 1e3, color="#eb6834", label="dense")
ax.set_xscale("log")
ax.set_xlabel("density: the fraction of the entries that are not zero")
ax.set_ylabel("milliseconds for A @ v")
ax.set_title(f"A @ v for N = {N:,}: sparse against dense, as the matrix fills")
ax.grid(True, which="major", color="#e1e0d9", linewidth=0.8)
ax.spines[["top", "right"]].set_visible(False)
ax.legend(frameon=False)
```

```
fig.tight_layout()
plt.show()
```

OUTPUT

```
dense product, 4,000 x 4,000 numbers: 2.64 milliseconds, 0.17 nanoseconds a number
  density    nonzeros   sparse product, ms  ns a nonzero   faster
    0.001      16,000                0.02          1.46   sparse
    0.003      48,000                0.04          0.85   sparse
     0.01     160,000                0.08          0.50   sparse
     0.03     480,000                0.17          0.35   sparse
      0.1   1,600,000                0.63          0.39   sparse
      0.3   4,800,000                2.46          0.51   sparse
        1  16,000,000                8.80          0.55    dense
```

```
CROSSOVER_DENSITY = next((d for d, s in zip(densities, sparse_seconds) if dense_seconds < s), None)
```

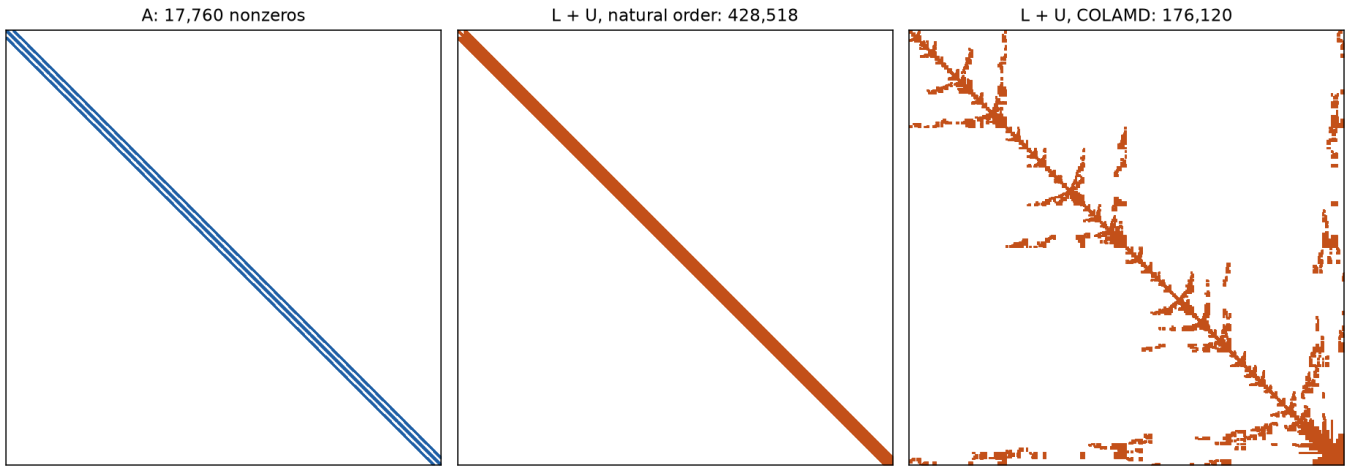
OUTPUT

```
Correct: from a density of 1 on, the dense product is faster on this machine.
```

Fill-in: the order of elimination

For a symmetric matrix, an off-diagonal nonzero $A[i,j]$ defines an edge between unknowns i and j. Eliminating an unknown updates its neighbours' equations and can create entries between them. These new entries are **fill-in**. In the symbolic pattern, the remaining neighbours become a clique: every pair is connected. Special numerical cancellations can remove some entries.

Elimination produces an upper-triangular matrix U. Its multipliers form a lower-triangular matrix L; these are the **LU factors**, with permutations accounting for reordering. The six-node star starts with 16 nonzeros. Eliminating its centre first gives 36 entries in the combined factor pattern; eliminating its leaves first leaves 16. The choice of elimination order therefore affects factor storage and arithmetic work.



The 60 x 60 grid's matrix, and its factors after elimination in the natural order and in SuperLU's default order.

For the 60×60 grid, SciPy's `splu` produces combined factor patterns with 24.1 times the original nonzeros in natural order, 9.9 times with COLAMD, and 5.9 times with a minimum-degree ordering. COLAMD is SuperLU's default column ordering. Reducing factor entries generally reduces storage and work, though elapsed time also depends on implementation details.

The grid-pattern table counts `(L + U).nnz`, with diagonal positions counted once. The timing table counts `L.nnz + U.nnz`, including both stored diagonals. The latter count is larger by N for these matrices.

Code: star orderings, grid factors, and solve times

cells `dl-star`, `dl-grid-fill`, `dl-grid-time`

```
import networkx as nx


def fill_pattern(A_):
    """Where elimination without pivoting leaves nonzeros: the pattern of the upper triangle after eliminate, plus the
    multipliers below it (the L part), as a 0/1 array."""
    n_ = A_.shape[0]
    B = A_.astype(float).copy()
    pattern = (B != 0)
    for k in range(n_):
        for i in range(k + 1, n_):
            if B[i, k] != 0:
                factor = B[i, k] / B[k, k]
                B[i, k:] -= factor * B[k, k:]
                pattern[i, k:] |= (B[k, k:] != 0)       # the row gains every nonzero of row k from k on
    return pattern


def star_matrix(order):
    """The matrix of a star with 6 nodes (centre and five leaves), rows and columns in the given order of the nodes."""
    S = np.eye(6) * 6.0
    for leaf in range(1, 6):
        S[0, leaf] = S[leaf, 0] = -1.0
    return S[np.ix_(order, order)]


first = fill_pattern(star_matrix([0, 1, 2, 3, 4, 5]))      # the centre eliminated first
last = fill_pattern(star_matrix([1, 2, 3, 4, 5, 0]))       # the centre eliminated last
print("nonzeros of the star matrix: 16;  after elimination, centre first:", int(first.sum()), " centre last:",
int(last.sum()))

G_star = nx.star_graph(5)
pos = nx.spring_layout(G_star, seed=2)
fig, axes = plt.subplots(1, 4, figsize=(12.5, 3.3))
nx.draw(G_star, pos, ax=axes[0], node_color=["#eb6834"] + ["#cfe1f2"] * 5, with_labels=True, node_size=380,
edge_color="#999999")
axes[0].set_title("the star: a matrix of 6 rows", fontsize=11)
G_clique = nx.complete_graph(range(1, 6))
nx.draw(G_clique, pos, ax=axes[1], node_color="#cfe1f2", with_labels=True, node_size=380, edge_color="#c6531a")
axes[1].set_title("the centre eliminated first:\nits neighbours all joined", fontsize=11)
axes[2].spy(first, markersize=9, color="#c6531a")
axes[2].set_title(f"centre first: {int(first.sum())} nonzeros", fontsize=11)
axes[3].spy(last, markersize=9, color="#1f5fa8")
axes[3].set_title(f"centre last: {int(last.sum())} nonzeros", fontsize=11)
for ax in axes[2:]:
    ax.set_xticks([])
    ax.set_yticks([])
```

```
fig.tight_layout()
plt.show()
```

OUTPUT

```
nonzeros of the star matrix: 16;  after elimination, centre first: 36  centre last: 16
```

```
A60 = grid_matrix(60).tocsc()
factors = {}
for name, spec in (("natural order", "NATURAL"), ("COLAMD (SuperLU's default)", "COLAMD"), ("minimum degree",
"MMD_AT_PLUS_A")):
    lu = sparse_linalg.splu(A60, permc_spec=spec)
    factors[name] = (lu.L + lu.U).tocsr()
print(f"the 60 x 60 grid: N = {A60.shape[0]:,}, {A60.nnz:,} nonzeros in A")
print(f"{'order of elimination':30s}{'nonzeros of L + U':>20s}{'times those of A':>18s}")
for name, F in factors.items():
    print(f"{name:30s}{F.nnz:20,d}{F.nnz / A60.nnz:18.1f}")

fig, axes = plt.subplots(1, 3, figsize=(12.5, 4.3))
axes[0].spy(A60, markersize=0.3, color="#1f5fa8")
axes[0].set_title(f"A: {A60.nnz:,} nonzeros", fontsize=11)
for ax, name in zip(axes[1:], ("natural order", "COLAMD (SuperLU's default)")):
    ax.spy(factors[name], markersize=0.3, color="#c6531a")
    ax.set_title(f"L + U, {name.split(' (')[0]}: {factors[name].nnz:,}", fontsize=11)
for ax in axes:
    ax.set_xticks([])
    ax.set_yticks([])
fig.tight_layout()
plt.show()
```

OUTPUT

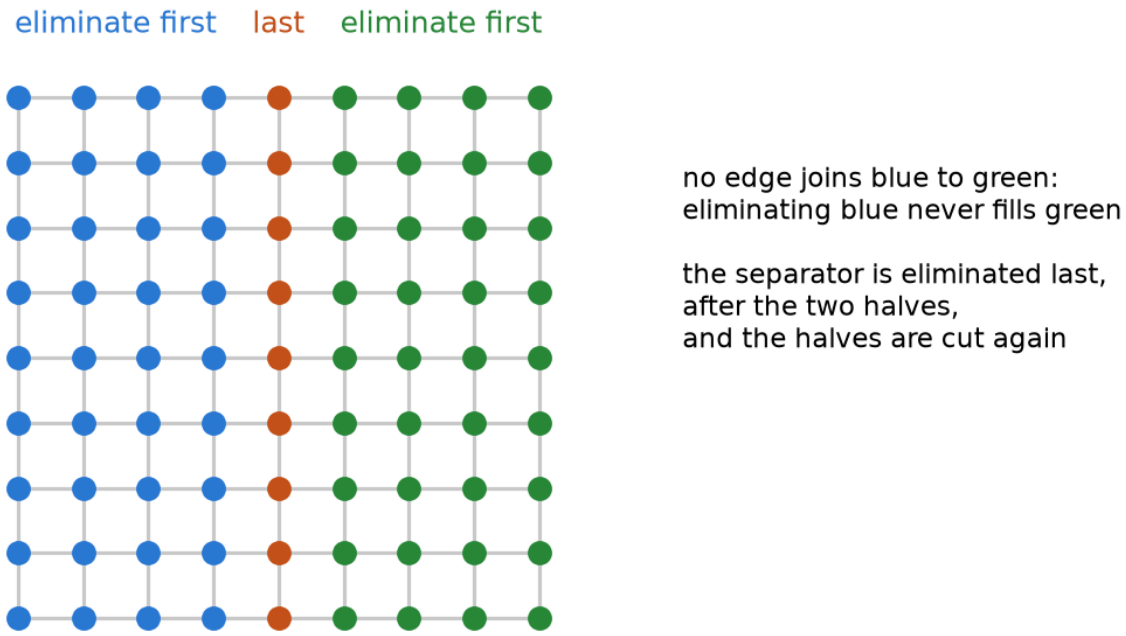
```
the 60 x 60 grid: N = 3,600, 17,760 nonzeros in A
order of elimination             nonzeros of L + U  times those of A
natural order                              428,518              24.1
COLAMD (SuperLU's default)                 176,120               9.9
minimum degree                             104,830               5.9
```

OUTPUT

```
      grid        N             order     nnz(L) + nnz(U)  seconds to solve
   80 x 80     6,400           natural           1,024,158             0.031
   80 x 80     6,400            COLAMD             370,282             0.008
  160 x 160   25,600           natural           8,192,318             0.387
  160 x 160   25,600            COLAMD           2,036,562             0.049
```

Natural ordering processes the unknowns in their original order. **Minimum degree** chooses a node with the fewest remaining neighbours. Eliminating a node of degree d can add at most $d(d-1)/2$ edges, so this greedy rule aims to limit immediate fill; it need not give the best overall ordering.

Nested dissection finds a **separator**, a set of nodes whose removal splits the graph into disconnected parts. It orders each part recursively and eliminates the separator last. While the separator remains, eliminating within one part creates no edges to the other. For a two-dimensional grid with N unknowns, nested dissection gives $O(N \log N)$ factor nonzeros and $O(N^{1.5})$ arithmetic work. Natural row-by-row ordering instead gives $O(N^{1.5})$ factor nonzeros and $O(N^2)$ work; dense elimination takes $O(N^3)$ work.



Nested dissection: the separator, in orange, is eliminated last, after the two halves it keeps apart.

Numerical pivoting seeks suitable pivot values; fill-reducing ordering seeks a sparse factor pattern. Nested dissection's advantage on large two-dimensional grids follows from its asymptotic bounds. It need not outperform every other ordering on a small grid.

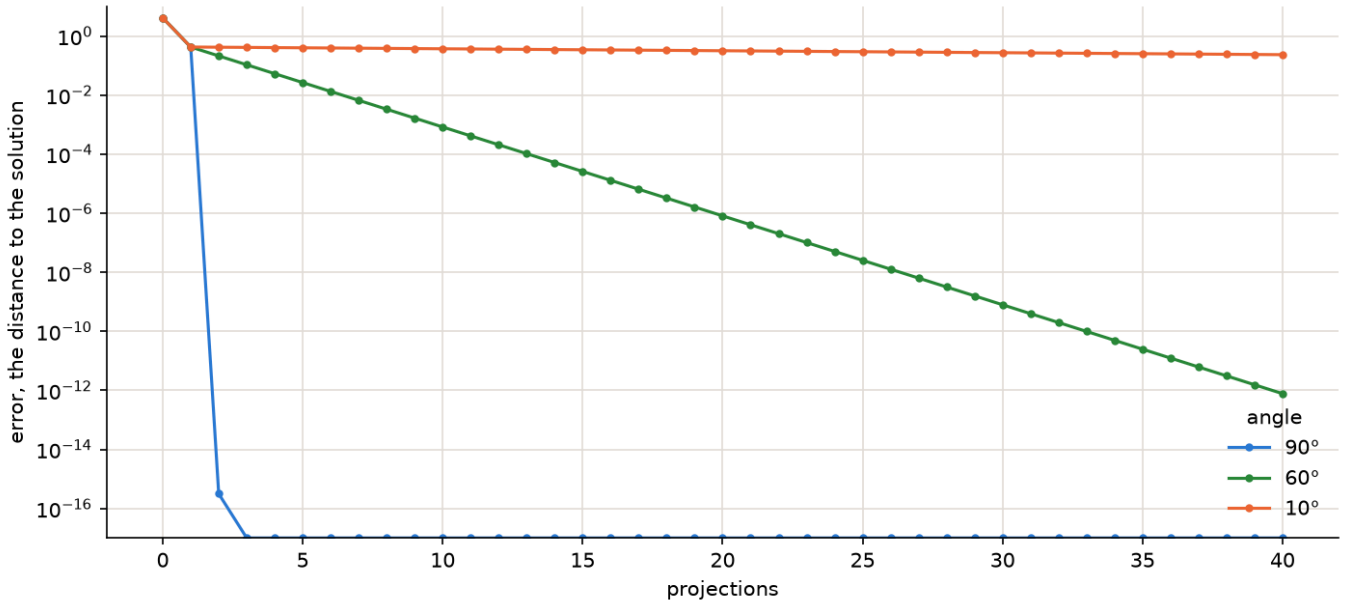
Solving by iterating

A **direct method** performs a finite sequence of operations that would produce a solution in exact arithmetic. An **iterative method** updates an approximation until a stopping criterion is satisfied. Kaczmarz's method, introduced in 1937, uses **orthogonal projection**: the nearest point on an equation's line or plane. For a real system, projecting onto row a_i with right-hand side b_i gives

$$x \leftarrow x - \frac{a_i \cdot x - b_i}{a_i \cdot a_i} a_i .$$

In exact arithmetic, the updated point satisfies equation i, although other equations may no longer hold. For a consistent system, projection cannot increase the distance to a solution on that plane. A sparse step uses the nonzeros of row i for a dot product and vector update.

For two intersecting lines, after the first projection, each complete cycle multiplies the distance to the intersection by $\cos^2\theta$, where θ is the angle between the lines. Two projections suffice at a right angle. The cycle factor is 0.25 at 60 degrees and about 0.97 at 10 degrees. Nearly parallel lines therefore require many more cycles for the same reduction in error.

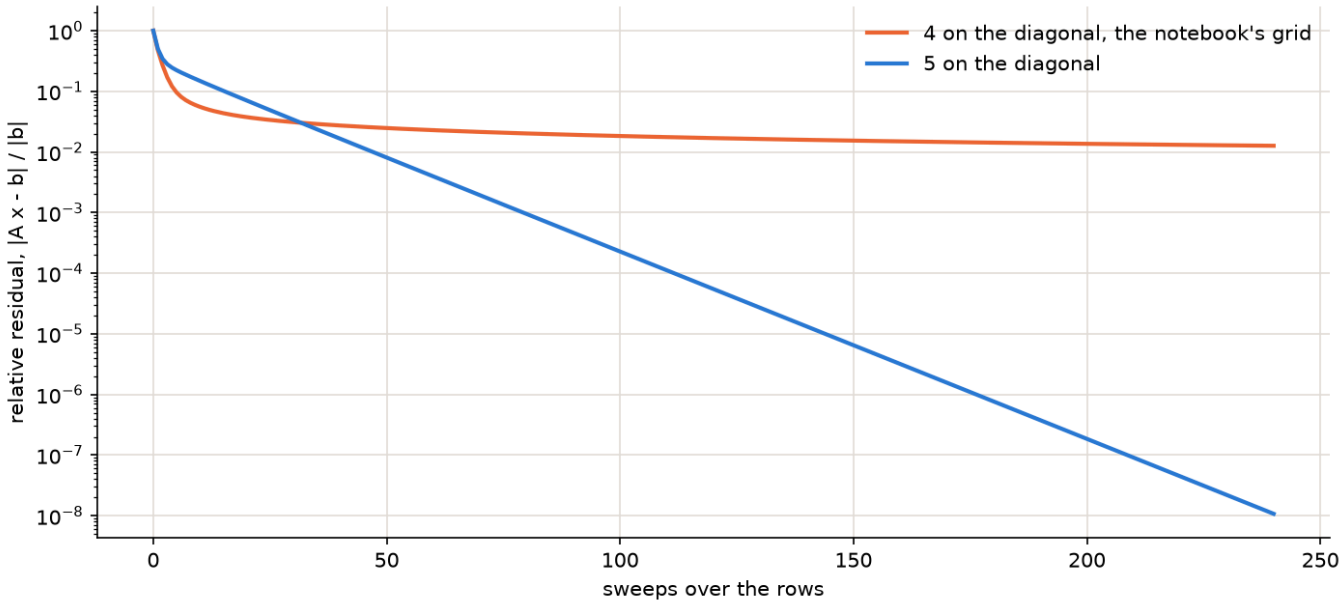


The error after each projection, at three angles: a fixed factor each cycle.

With row norms precomputed, a sweep of sparse Kaczmarz costs O(nnz) arithmetic work for these matrices. Total cost also depends on how many sweeps are needed. On the 40×40 grid with diagonal entries 4, the relative residual is still 1.3e-02 after 240 sweeps. Increasing the diagonal entries to 5 improves conditioning, and the residual falls below 10^{-6} after 177 sweeps. A residual tolerance provides a stopping criterion; conditioning determines what it implies about solution error. Finite precision can cause progress to stall.

Randomized Kaczmarz chooses each row with probability proportional to its squared Euclidean norm. For a consistent system with full column rank, Strohmer and Vershynin proved geometric convergence of the expected squared solution error. Their bound uses a scaled condition number involving the Frobenius norm, distinct from the default condition number returned by `np.linalg.cond`. Random selection can improve convergence, but does not guarantee fewer steps than cyclic ordering on every system. Their paper gives the sampling rule and bound.

Other iterative methods include Jacobi, Gauss–Seidel, and conjugate gradients. Their convergence conditions differ. For real matrices, `scipy.sparse.linalg.cg` requires symmetry and positive definiteness. These methods can use sparse matrix operations without constructing LU factors; an optional preconditioner may require additional storage and work.



Kaczmarz on two grid systems. Increasing the diagonal from 4 to 5 improves conditioning and speeds convergence in this experiment.

Code: the zigzag at three angles, the error against the projections, and the sweeps on the grid

cells `dk-kacz-2d`, `dk-kacz-rate`, `dk-kacz-grid`

```
def kaczmarz_path(rows, x0, steps):
    """The points visited by Kaczmarz's method on the rows [a1, a2, b], cycling through them."""
    x = np.array(x0, dtype=float)
    path = [x.copy()]
    for k in range(steps):
        a, b = np.array(rows[k % len(rows)][:2]), rows[k % len(rows)][2]
        x = x - (a @ x - b) / (a @ a) * a
        path.append(x.copy())
    return np.array(path)


def two_lines(angle_deg):
    """x + y = 2, and a second line through (1, 1) at the angle, with the same norm."""
    th = np.pi / 4 + np.radians(angle_deg)
    r2 = np.sqrt(2) * np.array([np.cos(th), np.sin(th)])
    return [[1.0, 1.0, 2.0], [r2[0], r2[1], r2[0] + r2[1]]]


START = (4.2, 3.6)
ANGLES = [90, 60, 10]
fig, axes = plt.subplots(1, 3, figsize=(13.5, 4.6))
for ax, deg in zip(axes, ANGLES):
    rows = two_lines(deg)
    xs = np.linspace(-1, 5, 2)
    for r, colour in zip(rows, ("#2a78d6", "#eb6834")):
        ax.plot(xs, (r[2] - r[0] * xs) / r[1], color=colour, linewidth=2.5)
    path = kaczmarz_path(rows, START, 12)
    ax.plot(path[:, 0], path[:, 1], "-", color="#1a1a1a", linewidth=1.4, alpha=0.8)
    ax.plot(path[:, 0], path[:, 1], "o", color="#1a1a1a", markersize=3.5)
    ax.plot(*START, "o", color="#999999", markersize=7)
    ax.plot(1, 1, "o", markerfacecolor="none", markeredgecolor="#CC0033", markersize=11, markeredgewidth=2)
    err = np.hypot(path[-1, 0] - 1, path[-1, 1] - 1)
```

```
    ax.set_title(f"{deg}° between the lines: after 12 steps the error is {err:.2g}", fontsize=11)
    ax.set_xlim(-1, 5); ax.set_ylim(-1, 5); ax.set_aspect("equal"); ax.grid(True, color="#e1e0d9")
    ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout()
plt.show()
```

```
fig, ax = plt.subplots(figsize=(9.5, 4.4))
print(f"{'angle':>7s}{'cos^2 of the angle':>20s}{'error after 2 steps':>21s}{'after 20':>10s}{'after 40':>10s}")
for deg, colour in zip(ANGLES, ("#2a78d6", "#2a8a3a", "#eb6834")):
    path = kaczmarz_path(two_lines(deg), START, 40)
    err = np.hypot(path[:, 0] - 1, path[:, 1] - 1)
    ax.semilogy(np.arange(len(err)), np.maximum(err, 1e-17), marker="o", markersize=3, color=colour, label=f"{deg}°")
    print(f"{deg:6d}°{np.cos(np.radians(deg)) ** 2:20.3f}{err[2]:21.2g}{err[20]:10.2g}{err[40]:10.2g}")
ax.set_xlabel("projections"); ax.set_ylabel("error, the distance to the solution")
ax.set_ylim(1e-17, 10)
ax.grid(True, which="major", color="#e1e0d9"); ax.spines[["top", "right"]].set_visible(False); ax.legend(frameon=False,
title="angle")
fig.tight_layout()
plt.show()
```

OUTPUT

angle	cos^2 of the angle	error after 2 steps	after 20	after 40
90°	0.000	3.1e-16	0	0
60°	0.250	0.21	8.1e-07	7.7e-13
10°	0.970	0.42	0.32	0.23

```
def kaczmarz_sweeps(A_csr, b, sweeps):
    """Cyclic Kaczmarz on a CSR matrix from x = 0: the relative residual after each sweep."""
    x = np.zeros(A_csr.shape[0])
    data, indices, indptr = A_csr.data, A_csr.indices, A_csr.indptr
    norms = np.array([data[indptr[i]:indptr[i + 1]] @ data[indptr[i]:indptr[i + 1]] for i in range(A_csr.shape[0])])
    history = [np.linalg.norm(A_csr @ x - b) / np.linalg.norm(b)]
    for _ in range(sweeps):
        for i in range(A_csr.shape[0]):
            r, idx = data[indptr[i]:indptr[i + 1]], indices[indptr[i]:indptr[i + 1]]
            x[idx] -= (r @ x[idx] - b[i]) / norms[i] * r
        history.append(np.linalg.norm(A_csr @ x - b) / np.linalg.norm(b))
    return np.array(history)


n_grid = 40
A4 = grid_matrix(n_grid).tocsr()                                # the notebook's grid: 4 on the diagonal
A5 = (A4 + sparse.identity(A4.shape[0], format="csr")).tocsr()    # the same, with 5: strictly diagonally dominant
x_true = rng.random(A4.shape[0])
SWEEPS = 240
hist = {name: kaczmarz_sweeps(A_, A_ @ x_true, SWEEPS) for name, A_ in (("4 on the diagonal, the notebook's grid", A4),
("5 on the diagonal", A5))}
N_ = A4.shape[0]
print(f"the {n_grid} x {n_grid} grid: N = {N_:,} unknowns, {A4.nnz:,} nonzeros; one sweep is {A4.nnz:,} multiply-adds, "
      f"elimination in the natural order about {N_ * n_grid ** 2 // 1:,}")
for name, h in hist.items():
    below = np.argmax(h < 1e-6) if (h < 1e-6).any() else None
    print(f"{name:40s} residual after {SWEEPS} sweeps {h[-1]:.1e}" + (f"; below 1e-6 after {below} sweeps" if below else
"; not yet below 1e-6"))

fig, ax = plt.subplots(figsize=(9.5, 4.4))
for (name, h), colour in zip(hist.items(), ("#eb6834", "#2a78d6")):
```

```
    ax.semilogy(np.arange(len(h)), h, color=colour, linewidth=2, label=name)
ax.set_xlabel("sweeps over the rows"); ax.set_ylabel("relative residual, |A x - b| / |b|")
ax.grid(True, which="major", color="#e1e0d9"); ax.spines[["top", "right"]].set_visible(False); ax.legend(frameon=False)
fig.tight_layout()
plt.show()
```

OUTPUT

```
the 40 x 40 grid: N = 1,600 unknowns, 7,840 nonzeros; one sweep is 7,840 multiply-adds, elimination in the
natural order about 2,560,000
4 on the diagonal, the notebook's grid   residual after 240 sweeps 1.3e-02; not yet below 1e-6
5 on the diagonal                        residual after 240 sweeps 1.1e-08; below 1e-6 after 177 sweeps
```

The attainable residual depends on the system, scaling, and computation. There is no universal residual floor of 10^{-16}, and a small residual alone does not guarantee a small solution error.